

传智杯 · AI WEB 网页开发挑战赛 · 技术文档

知行 Agent

会规划、会查证、会使用工具的透明化 AI 智能助手。本手册系统阐述项目的需求分析、系统架构设计、AI 技术方案、核心功能实现、测试与部署方案，并明确列出技术创新点与应用创新点。

赛道方向：AI Agent 智能助手 · 参赛组别：B组

· 2026 年 9 月

GITHUB.COM/SOLARWARD-1209/ZHIXING-AGENT

目录

1 项目概述	3
1.1 项目背景与选题方向	3
1.2 产品定位与核心价值	3
1.3 主要特性总览	3
1.4 开发与运行环境	4
2 需求分析	5
2.1 目标用户与使用场景	5
2.2 功能需求	5
2.3 非功能需求	5
3 系统架构设计	6
3.1 总体架构	6
3.2 事件协议设计	6
3.3 模块划分	7
3.4 关键设计决策	8
4 AI 技术方案	8
4.1 大模型集成（考察点权重 8%）	9
4.2 Agent 规划与行动循环（8%）	9
4.3 RAG 混合检索（6%）	9
4.4 多模态图片理解（5%）	10
4.5 AI 安全与可控性（4%）	10
4.6 MCP 风格工具集成（4%）	11
5 核心功能实现	11
5.1 执行计划可视化	11
5.2 流式输出与思考状态可视化	11
5.3 工具卡片与来源引用	11
5.4 会话持久化	11
5.5 演示模式设计	11
6 创新点说明	12

6.1 技术创新点	12
6.2 应用创新点	12
7 测试与部署	13
7.1 测试策略与用例	13
7.2 构建与运行	13
7.3 部署方案	13
8 总结与展望	14
8.1 当前成果	14
8.2 路线图	14
8.3 团队信息	15

1 项目概述

1.1 项目背景与选题方向

「知行 Agent」参加传智杯 AI WEB 网页开发挑战赛「AI Agent 智能助手」命题方向。该方向要求基于大模型构建具备自主规划、工具调用、多轮对话能力的智能体应用。当前大多数 AI 对话产品以“黑盒聊天框”形态呈现：用户看不到 AI 如何得出结论，无法判断答案可信度，也难以在需要精确计算或实时信息的场景中获得可靠结果。

本作品将「透明可查」作为产品核心：面对复杂问题时，Agent 先生成可见的执行计划，再检索内置知识库查证、按需调用外部工具求解，最后以流式方式输出带引用来源的回答，全过程在界面上实时可视化。产品口号“知你所问，行你所托”即来源于这一理念。

35%

评审权重中 AI 技术深度占比，本作
品六项考察点全部落点

15 篇

内置传智杯备赛知识库，覆盖官方规
则全部章节

38 项

vitest 单元测试用例，覆盖工具/检
索/安全/流水线

1.2 产品定位与核心价值

- 对用户：备赛学生可获得有据可查的赛事咨询（报名、组别、评分、提交、奖项），并使用计算、日期、天气等日常工具，回答附带引用编号与来源卡片，可信可查。
- 对评审：Agent 的每一步——理解、规划、检索、调用、生成——均在界面以时间线与卡片形式呈现，AI 能力深度与交互质量可直接观察验证。
- 对工程：前后端通过九类 SSE 事件协议解耦，检索层可插拔升级，工具层按统一契约注册，具备清晰的演进路径。

1.3 主要特性总览

特性	说明	对应评审考察点
执行计划可视化	问题自动拆解为 2~5 步计划，步骤状态实时更新	Agent/工作流设计（8%）
RAG 有据回答	BM25+向量混合检索，引用编号与来源卡片	RAG 技术应用（6%）
MCP 风格工具注册表	JSON Schema 契约统一管理工具，易于扩展	MCP/工具集成（4%）
图片理解	发送图片提问，视觉模型解读并结构化作答	多模态能力（5%）
安全兜底	输入风险拦截、专业建议免责、异常优雅降级	AI 安全与可控性（4%）
流式交互体验	打字机渲染、思考状态可视化、断流兜底	AI 交互体验（3%）
演示模式	未配置大模型 Key 时全功能本地演示	工程完整性与可验证性

表 1-1 主要特性与评审考察点对应关系

1.4 开发与运行环境

类别	选型	说明
前端框架	React 18 + Vite 5	组件化开发、HMR 快速迭代（官方推荐技术栈）
语言	TypeScript 5 (strict)	全量严格模式类型检查，零 any 逃逸
样式	Tailwind CSS 3	原子化 CSS（官方强烈推荐）
富文本渲染	react-markdown + remark-gfm	流式 Markdown 与表格渲染
服务端	Node.js ≥ 18 (TypeScript)	Agent 编排、SSE 推送、RAG 检索
大模型	DeepSeek / 智谱 GLM (OpenAI 兼容协议)	流式补全 + Function Calling
测试	Vitest 2	38 项单元测试，GitHub Actions 门禁
部署	Dockerfile / Render / Railway	常驻 Node 进程，公网在线演示

表 1-2 开发与运行环境

2 需求分析

2.1 目标用户与使用场景

目标用户为参加传智杯及类似技术竞赛的高校学生。备赛过程中的典型痛点包括：官方规则分散在多个页面，查找关键信息（截止日期、组别边界、评分权重）耗时且易遗漏；复杂数学计算依赖心算或外部工具；通用大模型对赛事规则一无所知且经常编造。本产品将"赛事规则问答"与"日常工具助手"合并为一个答案可信、过程透明的入口。

真实场景定义：回答必须"有据可查"，过程必须"看得见"，工具必须"算得准"。这三点同时命中实用性（10%）、AI 交互体验（3%）与 AI 安全可控（4%）三项评审标准。

2.2 功能需求

编号	功能	需求描述
FR-1	赛事知识问答	基于内置知识库回答报名/组别/评分/提交/奖项等问题，标注引用来源
FR-2	精确计算	四则运算表达式求值（含括号与取模），结果由本地工具求解
FR-3	日期时间查询	获取当前日期、星期、时间，支撑"截止日期倒推"类问题
FR-4	天气查询	主要城市天气查询（内置模拟数据，接口形态与真实数据源一致）
FR-5	图片理解	上传图片并针对图片提问，视觉模型解读作答
FR-6	多轮对话	携带历史上下文连续追问，会话可保存、切换与删除
FR-7	过程可视化	执行计划、工具调用、思考阶段全程实时展示

表 2-1 功能需求清单

2.3 非功能需求

类别	需求	实现策略
响应体验	首字延迟低，输出逐字呈现	SSE 流式推送 + 打字机渲染，规划与检索阶段状态可视化
可靠性	网络/模型异常不崩溃	统一异常兜底话术、断流检测、未完成事件的兜底收尾
安全性	拒绝违法违规请求，密钥不泄露	输入规则过滤、免责声明机制、密钥仅存环境变量
可扩展	工具与模型可插拔	JSON Schema 工具注册表、Embedder/Vision 适配器接口
可维护	改动可回归验证	38 项单元测试 + GitHub Actions 持续集成

表 2-2 非功能需求与实现策略

3 系统架构设计

3.1 总体架构

系统采用前后端分离的五层架构。前端负责渲染与交互，服务端负责编排与能力执行，两层之间以自定义的SSE 事件协议通信。该划分使 AI 能力的演进（换模型、加工具、升级检索）不触碰前端代码。

层级	组成	职责
表现层	React 组件 (Sidebar / MessageView / ChatInput)	会话管理、计划时间线、工具卡片、流式渲染、来源引用
协议层	shared/protocol.ts 九类事件	前后端共享类型契约，事件驱动渲染
编排层	server/agent.ts	理解→规划→行动（检索+工具）→生成 主循环，异常兜底
能力层	rag.ts / tools.ts / safety.ts / vision.ts	混合检索、工具执行、安全过滤、多模态
模型层	llm.ts (OpenAI 兼容客户端)	智谱 GLM / DeepSeek 流式补全与 Function Calling

表 3-1 分层架构与职责

3.2 事件协议设计

服务端把一次问答的完整执行过程拆分为九类事件，经 SSE (Server-Sent Events) 按序推送：

事件	载荷	前端行为
status	stage: understanding / planning / retrieving / tooling / answering / finished	顶部与气泡内思考状态文案
plan	steps: 字符串数组	渲染执行计划时间线
step_update	index、status、note	更新对应步骤为执行中/已完成并显示备注
tool_call	callId、name、args	插入"调用中"工具卡片
tool_result	callId、ok、result	按 display 类型渲染为计算/天气/知识库/通用卡片
token	content 增量	追加至回答文本，打字机效果
sources	KbSource 数组	渲染引用来源胶囊 (title + snippet 提示)
error	message	红色错误条提示，不影响后续兜底话术
done	meta: mode、model	结束流式状态，标注本次回答由哪个模式产出

表 3-2 SSE 事件协议

选择 SSE 而非 WebSocket 的原因：问答为"单请求、单向流"形态，SSE 基于普通 HTTP 即可承载，无需引入长连接状态管理与心跳保活，部署到任何反向代理后的兼容性也更好；停止生成通过中止 fetch 实现，语义清晰。

3.3 模块划分

模块	文件	关键实现
Agent 编排	server/agent.ts	规划 JSON 解析、知识上下文注入、≤3 轮工具循环、异常兜底
混合检索	server/rag.ts	BM25 打分、Embedder 适配器、0.45/0.55 融合、词项覆盖相关性判断
工具注册表	server/tools.ts	calculate 递归下降解析器、time、weather；JSON Schema 契约
视觉通道	server/vision.ts	GLM-4V 消息构建、输入校验（数量/体积/格式）、未配置降级
安全模块	server/safety.ts	风险话题规则、免责声明注入、兜底话术
演示模式	server/demo.ts	意图识别 + 真实执行检索与工具的本地流水线
接入层	server/api.ts、 server/main.ts	SSE 端点、健康检查、静态托管（生产模式）
会话客户端	src/hooks/useAgentChat.ts	SSE 解析、事件归约、localStorage 多会话持久化

表 3-3 模块与关键实现

3.4 关键设计决策

- 事件协议先行：先冻结九类事件契约再实现两侧，前端对"谁在产生事件"无感知——演示模式与真实大模型产生同一协议的事件，切换模型零前端改动。
- 检索层可插拔：Embedder 定义为接口，配置 Key 即启用向量通道，失败自动降级 BM25；检索层永不抛错，保证问答主流程可用性优先。
- 工具结果结构化：tool_result 携带 display 字段，前端据此把结果渲染为计算卡片、天气卡片等"生成式 UI"，而非千篇一律的文本。
- 演示模式即诚实模式：无 Key 时本地流水线真实执行检索与工具，回答由知识库原文组织，界面明确标注"演示模式"，不冒充大模型输出。

4 AI 技术方案

本章对应评审标准中权重最高的"AI 技术深度"（35%），逐项说明六类考察点的选型理由、集成方式与优化策略。

4.1 大模型集成（考察点权重 8%）

模型客户端实现 OpenAI 兼容协议，通过环境变量一键切换智谱 GLM 与 DeepSeek（当前演示环境为 deepseek-chat）。补全请求开启 stream 与 tools，SSE 增量解析器统一处理文本增量与 tool_calls 聚合（按 index 累积分片参数，兼容"finish_reason=tool_calls"与网关不发结束标志两种行为）。

- Prompt 设计：系统提示词明确"回答简洁结构化、引用标注 [n]、数学必用工具、不编造、拒绝违规"五条准则，并把检索到的知识片段以 [n] 编号注入上下文，使引用可溯源。
- 输出稳定性：temperature 按任务分档（规划 0.3 / 回答 0.6 / 图片理解 0.4）；规划输出强制 JSON 并用正则截取与默认计划双保险，规划失败不阻塞主流程。
- 安全：API Key 仅存环境变量（.env 已 gitignore），客户端永不接触密钥。

4.2 Agent 规划与行动循环（8%）

编排器实现"理解 → 规划 → 行动 → 生成"主循环。规划阶段由模型输出 JSON 计划（2~5 步），前端时间线实时呈现各步骤状态；行动阶段先对用户问题执行知识检索并把片段注入系统上下文，再进入最多 3 轮的流式工具循环：模型可自主选择 calculate / get_current_time / get_weather 等工具，编排器执行后把结果以 tool 消息回填，进入下一轮生成直至给出最终回答。

实测案例：提问"作品提交需要准备哪些材料？在线演示有什么要求？"时，模型自主决定同时调用知识库检索与时间工具，最终回答列出了四类材料要求，并基于当前日期推算出"距离提交截止约 50 天"——体现了自主规划与多工具协同决策能力。

4.3 RAG 混合检索（6%）

检索层采用"BM25 常驻 + 向量可插拔"的混合架构。BM25 参数 $k_1=1.5$ 、 $b=0.75$ ，中文文本使用字符 unigram+bigram 分词（无需外部分词器即可获得亚词级匹配能力）；标题与分类字段以 4 倍/2 倍权重参与索引，确保标题命中优先于正文偶然命中。

环节	策略	说明
打分	BM25 + 余弦相似度加权融合	向量可用时按 0.45 (BM25) / 0.55 (向量) 融合，仅 BM25 时独立打分
向量	Embedder 适配器（默认智谱 embedding-3）	文档向量启动时构建并常驻内存；查询向量实时计算
归一化	最终分数统一 max 归一化到 [0, 1]	不同检索模式下分数语义一致，前端展示可解释
相关性判断	词项覆盖率（top1 文档对查询词项的命中比例 ≥ 0.2 ）	与语料规模无关；覆盖率不足时如实告知未找到，不强行作答
引用溯源	片段编号注入 Prompt + 来源卡片	回答句子标注 [n]，界面渲染可悬停预览的来源胶囊

表 4-1 RAG 检索策略

- 选型理由：当前语料为 15 篇结构化规则文本，BM25 已可获得高质量召回且零外部依赖；向量通道以适配器形式就位，配置 Embedding Key 后自动启用，形成可演示、可对比、可演进的混合检索。
- 降级策略：Embedding 构建/调用任何失败均静默回退 BM25，检索层永不阻塞问答主流程。

4.4 多模态图片理解（5%）

图片输入走独立的视觉通道：前端以 data URL 上传（单图 $\leq 4\text{MB}$ 、一次 ≤ 2 张，服务端二次校验格式/体积/数量），服务端构建 OpenAI 兼容的 image_url+text 消息数组，调用视觉模型（默认智谱 GLM-4V-Flash）流式作答。视觉问答不混入 RAG/工具循环，避免无关工具调用；未配置视觉模型时返回清晰的启用引导而非报错，失败时优雅降级并提示更换图片。

4.5 AI 安全与可控性（4%）

防线	机制	示例
输入过滤	风险话题规则（赌博/色情/涉暴/涉毒/诈骗/攻击）命中即统一拒绝话术	拦截后不进入任何模型调用
输出免责	命中医疗/法律/投资类关键词时自动追加免责声明	AI 生成内容不构成专业建议
兜底降级	模型/网络异常统一转为友好话术 + error 事件	用户永远看到可理解的反馈而非堆栈
密钥治理	.env + .gitignore + .dockerignore 三重隔离，CI 内容扫描	公开仓库与镜像中无任何密钥

表 4-2 AI 安全四道防线

4.6 MCP 风格工具集成 (4%)

工具层以"注册表 + 契约"组织：每个工具声明 OpenAI/JSON Schema 函数契约（名称、描述、参数类型与必填项），编排器按协议调用 `executeTool` 并把结构化结果回填模型。新增工具只需登记 schema 与实现，编排器零改动；该接口形态与 MCP (Model Context Protocol) 的工具声明方式同构，接入真实 MCP Server 仅需一个协议适配层。

5 核心功能实现

5.1 执行计划可视化

前端以时间线组件渲染 `plan` 与 `step_update` 事件：每个步骤有待执行/执行中（旋转指示器）/已完成（对勾）三种状态，支持附注（如"召回 3 条候选，最高相关度 0.87"）。计划不是装饰——它真实反映编排器的执行进度，用户据此理解"AI 在做什么"。

5.2 流式输出与思考状态可视化

SSE 客户端以 `fetch + ReadableStream` 实现，按空行分帧、逐行解析 `data` 事件并归约到消息状态。思考阶段（理解/规划/检索/工具/生成）驱动跳动的状态文案；生成阶段以打字机效果渲染 Markdown (`react-markdown + gfm`)，流式光标以闪烁竖线提示。断流、HTTP 错误、主动停止三条路径均有明确的状态收尾，不会出现"永久转圈"。

5.3 工具卡片与来源引用

`tool_result` 的 `display` 字段驱动生成式 UI：计算结果渲染为等宽字体的"表达式 = 结果"卡片，天气渲染为城市/天气/温度/提示的结构卡片（并明确标注模拟数据），知识库检索渲染为召回摘要卡。来源胶囊悬停显示片段预览，与回答中的 [n] 编号一一对应，实现"结论可溯源"。

5.4 会话持久化

会话（含完整消息与工具卡片数据）自动保存至 `localStorage`：刷新或重开后恢复最近会话；侧边栏支持新建、切换、删除，标题取首条提问自动生成；空会话自动回收，存储异常静默降级。

5.5 演示模式设计

未配置大模型 Key 时，系统运行本地演示流水线：意图识别（赛事/计算/时间/天气）驱动执行计划，检索与工具真实执行，回答由知识库原文要点组织并附引用，界面顶栏与每条回答均明确标注"演示模式"。该设计让评审在无密钥环境下也能完整体验产品交互，同时恪守不冒充模型输出的诚实原则。

6 创新点说明

6.1 技术创新点

编号	创新点	说明
T1	全过程透明的 Agent 执行架构	将规划、检索、工具调用、生成拆解为九类 SSE 事件流，前端以时间线与卡片实时呈现——业界常见聊天框为黑盒形态，本作品把 Agent 的"思考路径"产品化
T2	BM25+向量双通道混合检索与词项覆盖相关性判断	零外部依赖的 BM25 常驻与可插拔向量适配器融合，相关性判断采用与语料规模无关的词项覆盖率，优于固定分数阈值
T3	演示模式与真实模式同构	同一事件协议下双模式运行，演示模式真实执行检索与工具、回答取自知识库原文并明确标注，兼顾可验证性与诚实性
T4	工具结果驱动的生成式 UI	tool_result 携带 display 语义，前端渲染计算/天气/知识库等专用卡片，超越纯文本回复的表现力

表 6-1 技术创新点

6.2 应用创新点

编号	创新点	说明
A1	"有据可查"的竞赛咨询范式	赛事规则问答强制引用溯源，直击通用大模型"幻觉"痛点，可平移至校园服务、企业客服等规则型场景
A2	备赛知识库与日常工具一体化的学习助手	规则问答、精确计算、日期推算、图片理解在同一透明界面完成，贴合学生备赛的真实 workflow
A3	无门槛体验设计	演示模式使评审与用户无需任何 API Key 即可体验完整交互，降低验证与使用门槛

表 6-2 应用创新点

7 测试与部署

7.1 测试策略与用例

项目使用 Vitest 构建 38 项单元测试，全部离线运行（不依赖网络与模型服务），覆盖五个维度：

维度	典型用例	数量
工具层	计算器 6 组精确求值、除零与非法输入不抛异常、天气模拟标注、Schema 契约完整性	10
检索层	三类赛事问题召回正确文档、分数区间与降序、覆盖率相关性正反例、来源映射	10
安全层	风险话题拦截、正常放行、医疗/投资免责追加	8
流水线	演示模式数学/赛事/风险三场景全事件流断言	3
多模态	图片校验、消息构建、未配置降级	7

表 7-1 测试用例分布

GitHub Actions 在每次 push 与 PR 时自动执行类型检查、单元测试与生产构建，主分支合入以全绿为门槛。

7.2 构建与运行

命令	用途
npm run dev	开发模式（Vite + /api 中间件，端口 5173）
npm run build	类型检查 + 生产构建（dist/）
npm run server	独立 Node 服务器（托管 dist 与 /api，端口 8787）
npm test	38 项单元测试

表 7-2 常用命令

7.3 部署方案

在线演示（评审加分项）要求公网可访问且评审期间服务可用。本作品提供独立 Node 服务器（server/main.ts），支持四条部署路线：

路线	适用	要点
Render（推荐）	零成本快速上线	Build: npm ci && npm run build; Start: npm run server ；健康检查 /api/health
Railway / Fly.io	同类 PaaS	流程一致，环境变量注入密钥
Docker	自有主机/云服务器	单阶段镜像，.env 不入镜像，密钥运行时注入
裸机 + pm2	已有服务器	pm2 守护 + Nginx 反向代理 + HTTPS

表 7-3 部署路线对比

部署后自检：/api/health 返回 ai 模式、首页可问答、引用来源正常渲染、平台日志无密钥、评审前一天确认服务存活。

8 总结与展望

8.1 当前成果

- 交付了具备自主规划、工具调用、RAG 查证、图片理解与安全兜底的完整 AI Agent Web 应用；
- 评审标准 AI 技术深度 35% 的六项考察点全部实现落点，其中大模型集成、Agent 循环、RAG、安全与工具集成已可实测试验证；
- 工程侧达成 TypeScript 严格模式零错误、38 项测试全绿、CI 门禁、四路线部署预案；
- 公开仓库（含 README、架构说明与提交历史）满足大赛对代码仓库的全部要求。

8.2 路线图

方向	内容	价值
检索存储化	向量入库（pgvector/Milvus）+ 文档上传与自动分块	支撑更大语料与用户自定义知识库
真 MCP 接入	以 MCP 客户端接入外部 Server，实现工具动态发现	从"自建工具"走向"生态工具"
端侧智能	Transformers.js / WebGPU 浏览器端小模型推理	低延迟、隐私友好的端侧能力
语音交互	Web Speech API 语音输入与朗读	降低输入门槛，增强无障碍体验

表 8-1 演进路线图

8.3 团队信息

项目	内容
队伍名称	_____（提交前填写）
成员及分工	_____（提交前填写）
指导老师	_____（提交前填写）
参赛组别	B组（非 985/211 普通本科院校本科生）
代码仓库	<code>github.com/Solarward-1209/zhixing-agent</code>

表 8-2 团队信息

感谢大赛组委会提供的规则资料与备赛资源。本作品将持续迭代，欢迎评审专家通过在线演示地址体验完整功能。